

MSA 실습 프로젝트 이해 - 온라인 교육 플랫폼

4반 U110 김근홍

Chapter 1. 고객 중심 서비스 이해

1. 서비스 개요

본 프로젝트는 Gateway, Eureka, Kafka, MariaDB를 이용한 온라인 교육 플랫폼입니다. 온라인 교육을 듣고 싶어하는 학생과 자신의 지식을 제공할 프리랜서 혹은 선생님들이 불편함 없이 이용하는데 초점이 맞춰져 있습니다. 누구나, 고가의 장비가 없어도 지식을 나누고 습득할 수 있는 시스템을 목표로 설계되었습니다.

2. 고객에게 제공하는 가치

- **쉬운 결제 시스템:** 결제 구조를 단순화하여 원하는 교육을 쉽고 빠르게 결제할 수 있습니다.
- **강의 제공 간편화:** 강의 제공자는 자신이 만든 강의를 동영상 업로드만 신경쓰고, 이후 작업은 자동으로 처리하는 방식을 제공합니다.

3. 주요 고객 적용 사례

- **프리랜서 개발자:** 자신이 원하는 강의를 언제, 어디서든 제공할 수 있습니다.
- **새로운 분야를 배우고 싶은 누구나:** 자신이 잘 몰랐던 분야나 새롭게 배우고 싶은 분야를 온라인을 통해 시간적, 공간적 제약없이 배울 수 있습니다.

4. 고객이 느낄 수 있는 한계

- **MariaDB의 확장성 한계:** 대량 트랜잭션 발생 시 성능 저하 우려.
- **강의 다양화 부족:** 한정된 카테고리의 강의만 제공될 위험이 있음.

5. 구조 전환 제안

- **데이터베이스 확장 (PostgreSQL / Oracle)**
고속 처리와 다중 연결에 최적화된 DB로의 교체 또는 Read Replica를 도입하면, 데이터 규모가 커져도 안정성을 유지할 수 있습니다.
- **외부 강사/플랫폼 연동 API 구축 (Open API 제공)**

자사 플랫폼에 직접 등록되는 강의 외에도, 외부 교육 콘텐츠 제공사(CP)나 유튜브·인프런 등 타 플랫폼의 외부 강의 데이터를 연동/수집할 수 있는 API 인프라를 구축하여 빠르게 카테고리를 확장합니다.

Chapter 2. 시스템 기술 구조 및 설계 이유

1. Gateway 라우팅이 필요한 이유

복잡하게 분산된 백엔드 서비스들을 하나로 묶어 클라이언트에 단순한 인터페이스를 제공하고, 인증·보안·로드밸런싱 등 공통 기능을 일괄 관리하여 개발 생산성과 시스템 안정성을 높이기 위해 필요하게 되었습니다.

2. Eureka를 선택한 이유

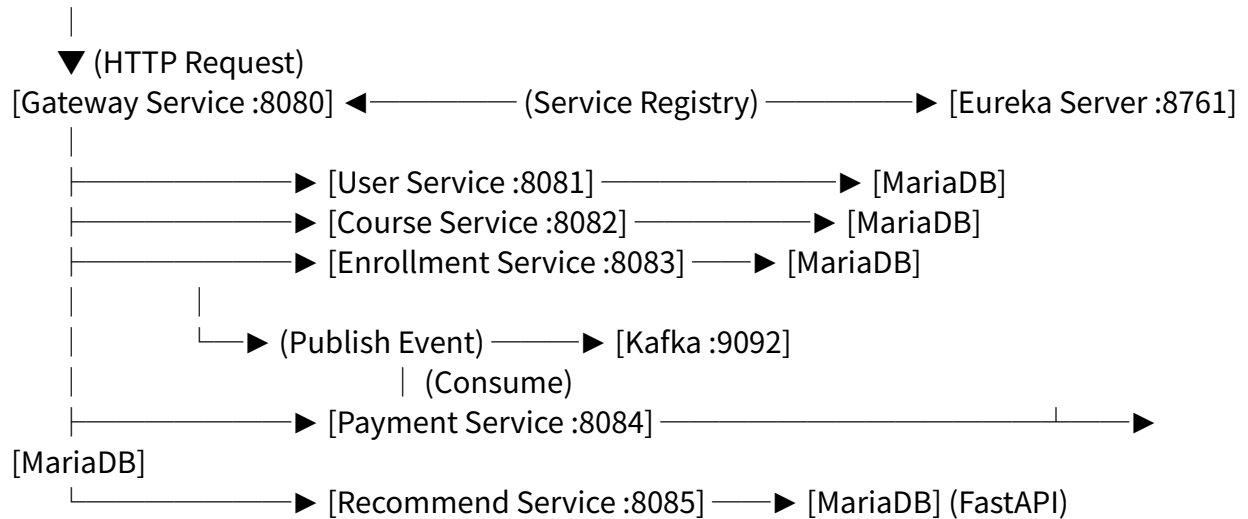
- **동적 IP 및 포트 관리:** 클라우드 및 오토스케일링 환경에서 수시로 변경되는 서비스의 IP와 포트 정보를 중앙에서 자동으로 등록하고 관리합니다.
- **서비스 위치의 하드코딩 방지 및 결합도 완화:** IP 주소 대신 서비스 이름 기반으로 통신하여 위치 의존성을 제거하고 코드 변경 없이 시스템을 유연하게 확장할 수 있습니다.
- **부하 분산 지원:** 동일 서비스의 여러 인스턴스 목록을 클라이언트에 전달하여 요청을 효율적으로 분산 처리(Load Balancing)하도록 돕습니다.
- **상태 감지 및 장애 격리:** 주기적인 Heartbeat 검사로 이상이 생긴 인스턴스를 서비스 목록에서 자동 제외하여 시스템의 안정성과 가용성을 유지합니다.

3. 시스템 구조 및 요청 처리 흐름

컨테이너	이미지	포트	의존성
gateway	SpringBoot 빌드	8080	eureka
eureka	SpringBoot 빌드	8761	-
user-service	SpringBoot 빌드	8081	eureka, mariadb
course-service	SpringBoot 빌드	8082	eureka, mariadb
enrollment-service	SpringBoot 빌드	8083	eureka, mariadb, kafka
payment-service	SpringBoot 빌드	8084	eureka, mariadb

recommend-service	FastAPI 빌드	8085	eureka, mariadb
kafka	confluentinc/cp-kafka	9092	-
mariadb	mariadb:latest	3306	-

[Client / Browser]



클라이언트 요청 처리 흐름:

1. 클라이언트가 Gateway(:8080)로 API 요청 전송
2. Gateway는 Eureka Server에서 각 서비스의 실시간 IP/Port 위치를 조회하여 요청을 포워딩[cite: 2, 3]
3. 각 마이크로서비스는 MariaDB에서 데이터를 처리하며, 수강 신청/결제 등의 상태 변화 발생 시 Kafka 메시지 브로커로 이벤트를 발행/수신하여 비동기 데이터 동기화 수행

Chapter 3. 트러블슈팅 사례 정리

구분	문제 상황 (오류)	원인 및 해결 방안
Docker build error	동시 빌드 시 Docker Hub Rate Limit (접속 제한) 발생	원인: 동일 IP 대역에서의 무인증 Pull 요청 한도 초과 해결: ① Docker Hub 계정 로그인 후 빌드 진행 ② 로컬 캐시 이미지 활용 ③ 테더링을 통한 IP 분산 처리
카테고리 등록 에러	카테고리 중 'AI' 선택 후 강의 생성 시 500 Internal Server Error 발생	원인: 프론트엔드 카테고리 Enum 값과 백엔드 데이터베이스 카테고리 매핑 누락 해결: 프론트엔드 카테고리 옵션에 'AI' 라벨 추가 및 API 데이터 페이로드 검증 로직 보완

✓ 트러블슈팅 핵심 요약

- 동시 빌드 환경에서는 IP 기반 Rate Limit에 걸리지 않도록 Docker 인증 및 로컬 이미지 캐싱 전략을 수립해야 합니다.
- 프론트엔드와 백엔드 간 Data Enum(카테고리 등) 불일치는 500 에러를 야기할 수 있으므로, API 요청/응답 스키마 검증(Pydantic/DTO)을 명확히 해야 합니다.